Research Assistant

Individual Local Release Report, User Manual, and Future Extension Plan

Prepared for Release Review

April 29, 2026

Contents

Chapter 1

Executive Summary

1.1 Current release target

The current 'research-assistant' release is an individual local research tool. It is designed for one researcher working in a private filesystem workspace, with offline-by-default workflows and Git-based sharing between researchers.

The release is not a shared department server, not a database-backed collaboration system, not an SSO/RBAC platform, and not a hosted user interface. Those capabilities are future extensions.

1.2 What has been built

The implemented tool provides:

- local workspace initialization, validation, repair, and configuration;
- privacy and offline-readiness reporting;
- demo setup and demo run workflows;
- paper ingestion from arXiv IDs, local PDFs, DOI-like queries, and title or topic queries;
- source-first arXiv LaTeX inspection when source is available;
- parser-tool readiness diagnostics and fixture-only parser benchmark smoke checks;
- review notes, derivation worksheets, experiment records, synthesis proposals, traceability reports, and governance scaffolds;
- backup, inspect, and restore workflows with dry-run and confirmation safeguards;
- repository hygiene checks for shareable Git repositories;
- workspace merge dry-run, explicit apply, provenance preservation, and post-merge rebuild;
- individual Git release validation records, validation reports, fixture rehearsals, performance rehearsals, and release gates;
- release reports and release artifact manifests.

1.3 Release status

Local validation supports a limited individual pilot. Broad individual or Git-shared release remains blocked until the following manual gates are completed:

- real fresh-reader onboarding from the documentation;
- real macOS validation;
- real minimal parser-tool machine validation;
- explicit release-owner approval for tag creation;
- explicit release-owner approval for artifact publication.

The release gate intentionally reports those blockers. They must not be waived or marked as passed unless the real evidence exists.

Chapter 2

Product Model

2.1 One researcher at a time

The core product assumption is simple: one researcher runs the tool locally on a private workspace. The workspace is a directory containing configuration, papers, source records, summaries, reviews, derived artifacts, indexes, backup archives, and governance records.

This makes the tool easy to inspect, easy to back up, and easy to debug. It also avoids requiring a database administrator, service operator, identity provider, or shared hosting environment for the current release.

2.2 Git as the sharing layer

Sharing is done by Git. A researcher may publish or exchange a repository that contains only shareable artifacts. Another researcher can clone that repository, inspect it, run repository hygiene checks, dry-run a workspace merge, apply a safe subset only with explicit confirmation, and rebuild derived artifacts.

Git is not treated as a transactional database. Merge conflicts and research disagreements remain human review events.

2.3 Trust boundary

Generated outputs are review material. Parser outputs, benchmark results, derivation worksheets, experiment records, synthesis proposals, traceability reports, readiness reports, and validation records do not certify mathematical correctness, scientific parser accuracy, or research approval.

Accepted human conclusions must stay explicit. The tool must not silently promote another researcher's generated artifact into accepted local conclusions.

Chapter 3

Workspace And Artifact Model

3.1 Local directories

A typical initialized workspace contains:

Path	Purpose
'.research-assistant/'	local configuration
'local_research/papers/raw/'	private raw PDFs and paper files
'local_research/papers/extracted/'	extracted text and parser output
'local_research/papers/source/'	cached source bundles and source records
'local_research/metadata/'	metadata records
'local_research/summaries/'	paper summary records
'local_research/reviews/'	review records
'local_research/analysis/'	derivations, synthesis, traceability, reports
'local_research/experiments/'	experiment plans and run records
'local_research/governance/'	validation, release, model policy, and gate records
'local_research/indices/'	rebuildable indexes
'local_research/exports/'	exports and backups

3.2 Shareable and private files

The release intentionally separates shareable research artifacts from private or generated local state.

Shareable examples include sanitized summaries, metadata, source records, links, reviews, derivation worksheets, experiment records, traceability records, synthesis proposals, benchmark manifests, model policies, and citation graph reports.

Private or non-shareable examples include raw PDFs, extracted paper text, backup archives, credentials, provider keys, tokens, '.codex', '.claude', caches, bytecode, 'build/', and 'dist/'.

Chapter 4

Command Manual

4.1 Install and initialize

```
ra --help
ra version
ra init
ra doctor
ra doctor --matrix
```

'ra init' creates an idempotent local workspace. 'ra doctor' reports optional tool status, offline status, platform status, and workflow readiness.

4.2 Demo workflow

```
ra --root /tmp/research-assistant-demo init
ra --root /tmp/research-assistant-demo demo setup
ra --root /tmp/research-assistant-demo demo run
ra --root /tmp/research-assistant-demo release-report
```

The demo creates a small synthetic research workflow with review material, derivation, experiment, traceability, governance, and backup artifacts.

4.3 Source and parser inspection

```
ra source-fetch --arxiv-id 2401.00001
ra source-show --paper-id paper_example
ra source-sections --paper-id paper_example
ra source-equations --paper-id paper_example
ra source-theorems --paper-id paper_example
ra parser-tool-matrix
ra parser-benchmark-smoke
ra parse-pdf --pdf /path/to/paper.pdf
```

Source inspection is strongest when arXiv LaTeX source is available. PDF parser workflows are useful for inspection but parser scientific accuracy is not certified.

4.4 Review and research artifacts

```
ra find --query "transport maps"
ra show --paper-id paper_example
ra review-list
ra review-show --paper-id paper_example
ra review-mark --paper-id paper_example --status approved
ra derivation create --paper-id paper_example --title "Derivation worksheet"
ra experiment create --paper-id paper_example --claim-id claim_1 --checklist-id
    reproducibility
ra synthesis propose --paper-id paper_example
ra traceability build --paper-id paper_example
```

These commands create or inspect review material. They do not by themselves create accepted mathematical conclusions.

4.5 Backup and restore

```
ra backup create
ra backup inspect --path /path/to/backup.tar.gz
ra --root /tmp/restore-check backup restore --path /path/to/backup.tar.gz
ra --root /tmp/restore-check backup restore --path /path/to/backup.tar.gz --no-dry-run
     --confirm-restore
```

Restore is dry-run by default. A real restore requires confirmation. Overwrite requires an additional opt-in.

Chapter 5

Git Sharing Workflow

5.1 Repository hygiene

```
ra repository-hygiene policy
ra repository-hygiene classify local_research/summaries/example.json
ra repository-hygiene check --strict
```

Strict hygiene checks Git state, private/generated path classes, and secret-like payload fields. It is intended to run before publishing or sharing a research repository.

5.2 Merge dry-run and apply

```
ra --root /path/to/my/workspace workspace merge --source /path/to/other/repo --dry-run
ra --root /path/to/my/workspace workspace merge --source /path/to/other/repo --apply
    --confirm-merge
ra --root /path/to/my/workspace workspace rebuild-derived
```

Dry-run is the default. Apply refuses to proceed without explicit confirmation, and conflicts involving accepted review material remain blockers.

5.3 Provenance

Applied imports preserve provenance showing the source repository, source Git commit when available, merge report ID, and merge timestamp. This makes later review possible without pretending that imported material is locally accepted.

Chapter 6

Nontrivial Showcases

6.1 DSGE survey chapter from a NeuTra seed

A realistic adoption story is not "the tool writes a paper." A better story is: a researcher needs to write a DSGE survey chapter on normalizing-flow and neural-transport architectures for difficult posterior geometry, and wants an auditable workspace rather than a folder of disconnected PDFs, notes, code experiments, and partially remembered assistant conversations.

One useful seed paper for this workflow is "NeuTra-lizing Bad Geometry in Hamiltonian Monte Carlo Using Neural Transport." From that seed, the researcher can build a chapter map around:

- transport maps and neural reparameterizations for posterior geometry;
- planar, radial, autoregressive, coupling, spline, and continuous normalizing-flow architectures;
- the relationship between flow expressiveness, Jacobian cost, and HMC or variational-inference workflows;
- which methods look directly relevant to DSGE posterior sampling and which are only generally relevant machine-learning background;
- criticisms, failure modes, diagnostics, and remedies reported by later citing papers;
- code experiments or derivation notes needed before a method can be trusted in a chapter.

6.1.1 Workspace setup

The researcher starts by creating a local workspace and ingesting a small public seed set. The exact paper identifiers may vary by metadata source, so the first pass is treated as review material.

```
ra init
ra ingest --query "NeuTra-lizing Bad Geometry in Hamiltonian Monte Carlo Using Neural
    Transport"
ra ingest --query "Normalizing Flows for Probabilistic Modeling and Inference"
ra ingest --query "Riemann Manifold Langevin and Hamiltonian Monte Carlo Methods"
ra find --query "neural transport HMC"
ra show --paper-id neutra_hmc
```

The important behavior is not that the first summary is perfect. The important behavior is that the metadata, summary, provenance, extraction status, and limitations are inspectable before the paper is trusted.

6.1.2 Literature graph expansion

After the seed paper is in the local store, citation commands help replace manual graph traversal. A reviewer can inspect nearby work, not blindly accept it.

```
ra citation-neighborhood --paper-id neutra_hmc --limit 25
ra discover --query "normalizing flows posterior geometry HMC DSGE"
ra discover --query "transport maps Bayesian inference Hamiltonian Monte Carlo"
```

A useful output for the chapter is a first-pass table with columns such as: architecture family, representative paper, inference target, claimed benefit, cost or limitation, DSGE relevance, and review status. This table can live in a chapter draft, a review note, or an exported context packet. The package's job is to make the evidence trail durable: every row should point back to a paper record, citation neighborhood, source section, derivation worksheet, or code experiment.

6.1.3 Architecture taxonomy as review notes

The researcher can record a provisional taxonomy without pretending it is final truth. For example:

```
ra audit-note append --paper-id neutra_hmc --field open_questions \
  --value "Does the learned transport remain stable for DSGE posterior ridges?"
ra audit-note append --paper-id neutra_hmc --field relevant_sections \
  --value "Use as neural-transport seed for normalizing-flow survey chapter."
ra audit-note append --paper-id normalizing_flows_review --field claimed_results \
  --value "Classify flow families by invertibility, Jacobian cost, and expressiveness.
      "
```

Those notes are intentionally review notes. They guide the chapter but do not become accepted technical conclusions until the researcher approves them.

6.1.4 Connecting papers to chapter sections and code

A chapter usually fails when the literature, derivations, experiments, and text drift apart. The link workflow keeps those references explicit.

```
ra link-add --paper-id neutra_hmc \
  --target docs/chapters/dsge_normalizing_flows.tex \
  --relationship "supports chapter section on neural transport HMC" \
  --target-type chapter_section --target-ref "sec:neutra"

ra link-add --paper-id neutra_hmc \
  --target experiments/dsge_neutra_posterior_geometry.py \
  --relationship "motivates posterior-geometry experiment" \
  --target-type code
```

```
ra derivation create --paper-id neutra_hmc \
  --title "Transport reparameterization and HMC geometry checklist"
ra synthesis propose --paper-id neutra_hmc --kind "survey_chapter_outline"
ra traceability build --paper-id neutra_hmc
```

The result is not a finished chapter. The result is a workspace where a colleague can ask: "Which paper supports this claim, what assumptions were recorded, what experiment checks it, and what still needs human review?"

6.1.5 Exporting chapter context

Once a subset of records has been reviewed, the researcher can export context for writing in another tool.

```
ra review-mark --paper-id neutra_hmc --status approved
ra export-context --review-status approved --output /tmp/dsge_flow_chapter_context.
    json
```

This export is useful input for a writing session, a code-generation session, or a colleague handoff. It is also deliberately conservative: only explicitly approved records should be used as trusted context, and even approved records remain traceable to source artifacts.

6.1.6 Why this attracts colleagues

For a colleague, the appeal is practical. The package shortens the path from "I have a seed paper and a vague chapter topic" to:

- a local literature workspace;
- a visible citation neighborhood;
- a provisional but reviewable architecture taxonomy;
- paper-to-chapter and paper-to-code links;
- derivation and experiment checklists;
- an exportable context packet for writing;
- explicit uncertainty and review state.

That is the honest adoption claim: the tool reduces context loss and manual literature bookkeeping for serious technical writing. It does not replace the researcher's judgment.

6.2 Limited-pilot release packet

```
ra --root /tmp/research-assistant-final-release init
ra --root /tmp/research-assistant-final-release release-report
ra --root /tmp/research-assistant-final-release repository-hygiene check --strict
ra --root /tmp/research-assistant-final-release individual-git-release validation-
    substitutes
ra --root /tmp/research-assistant-final-release individual-git-release fixture-
    rehearsal
```

```
ra --root /tmp/research-assistant-final-release individual-git-release performance --
    tier synthetic_git_1000 --synthetic-count 1000 --timeout-seconds 600
ra --root /tmp/research-assistant-final-release individual-git-release validation-
    report
ra --root /tmp/research-assistant-final-release individual-git-release gate-build
```

This demonstrates that local fixture evidence can be complete while broad release remains blocked for real external validation and approval.

6.3 Clean install from exact wheel

```
scripts/build_release_artifacts.sh
WHEEL_PATH=dist/research_assistant-0.1.0-py3-none-any.whl scripts/
    run_clean_install_smoke.sh
```

The explicit 'WHEEL_PATH' avoids accidentally installing whichever wheel sorts last in 'dist/'.

6.4 Parser degradation inspection

```
ra doctor --matrix
ra parser-tool-matrix
ra parser-benchmark-smoke
```

These commands show what parser workflows are available on the local machine. They are availability and fixture checks, not scientific parser certification.

Chapter 7

Validation Evidence

7.1 Local tests

The release validation packet includes:

- focused individual release integration tests;
- industrial scaffold regression tests;
- ‘scripts/run_fast_tests.sh’;
- ‘scripts/run_bounded_tests.sh’;
- packaging and artifact build smoke;
- explicit wheel clean-install smoke;
- individual Git release gate script;
- clean-checkout gate reproduction.

7.2 Representative performance

The current local evidence includes synthetic Git-sharing performance through the ‘synthetic_git_1000’ tier. This validates mechanics at that synthetic size. It does not certify performance on private real corpora.

7.3 Known blocked gates

The release gate remains blocked for:

- fresh-reader onboarding not yet recorded;
- macOS validation not yet recorded;
- minimal parser-tool machine validation not yet recorded;
- release-owner tag approval not yet recorded;
- release-owner publication approval not yet recorded.

Chapter 8

Maintainer Notes

8.1 Where to look

Maintainers should start with:

- ‘docs/maintainer_guide.md’;
- ‘docs/proposal/individual_git_release_target.md’;
- ‘docs/release_checklist.md’;
- ‘src/research_assistant/cli.py’;
- ‘src/research_assistant/individual_release.py’;
- ‘src/research_assistant/individual_git_release.py’.

8.2 High-risk changes

Treat these changes as release-critical:

- changing backup restore confirmation semantics;
- changing merge dry-run or apply confirmation behavior;
- changing forbidden path or secret scanning policy;
- changing validation scopes or gate readiness flags;
- weakening parser limitation language;
- changing JSON schema versions or artifact IDs.

Chapter 9

Known Limitations

- The current release is local and file-based.
- Git sharing is not real-time collaboration.
- Parser scientific accuracy is not certified.
- Generated artifacts remain review material.
- Real external platform and human onboarding evidence is still missing.
- Tagging and publication require explicit release-owner approval.
- Multi-user database/service/RBAC/hosted UI work is future scope.

Chapter 10

Future Multi-User Extension

The future extension path may include:

- shared storage with migration policy;
- service deployment and background jobs;
- SSO/RBAC and audit-log policy;
- hosted UI for inspection and review;
- department operations and support SOPs;
- stronger parser/source benchmarks on gold corpora;
- real-world scalability validation on sanitized corpora;
- governed live-provider or LLM integration.

Those are not prerequisites for the current individual release. They should be planned and validated separately so the local tool can remain simple, inspectable, and useful.

Release Checklist Summary

```
scripts/run_fast_tests.sh
scripts/run_bounded_tests.sh
PYTHONPATH=src python -m pytest tests/integration/test_individual_release_cli.py -q
scripts/build_release_artifacts.sh
WHEEL_PATH=dist/research_assistant-0.1.0-py3-none-any.whl scripts/
    run_clean_install_smoke.sh
scripts/run_individual_git_release_gate.sh
git diff --check
git status --short --ignored
```

Final broad release requires real external validation and release-owner approval. Until then, the tool is suitable for limited individual pilot use with documented limitations.